

BE202 Final Project: Glove Mouse

Xie, Wen-hao (1230021133) and Shi, Zhi-bo (1230017906)

EX04, Group 4, BSAS BEIE, MUST

December 11, 2024

Abstract

This project describes the design of a glove mouse, which is an alternative to computer input devices. Using Arduino Leonardo, combined with the LSM9DS0 accelerometer and other sensors. Designed to improve the mobility and ease of use of the mouse. The realization process from hardware to software is described. It shows potential in a variety of applications, especially for people with physical disabilities.

1 Background

The development of a wearable, gesture-based glove mouse represents a significant advance in human-computer interaction and aims to improve the user experience and accessibility of modern computing. Traditional mechanical mice often rely on repetitive and precise movements that may not be intuitively felt by all users, especially those with physical disabilities or limitations, while including those who may struggle to master fine motor skills or experience discomfort from prolonged use of standard equipment. To address these challenges, gesture-based devices have emerged as innovative solutions that leverage natural movements to create more convenient interactions.

The glove mouse provides a more intuitive and user-friendly alternative to traditional input methods. This approach not only simplifies the interaction process, but also has the potential to make computers easier to use for a wide variety of users. At the same time, the glove mouse also provides the user with more precise control in various aspects such as design and art.

In this project, we made a glove mouse using Arduino Leonardo and acceleration sensors



Figure 1: An example of glove mouse

2 Principle

The basic mechanism of the glove mouse relies on sensors to recognize hand movements and convert mechanical motion signals into digital signals to control mouse movements. [1][2] The 9-axis MEMS sensor module picks up the hand movement and transmits it through data to the computer, as shown in Figure 2b. This signal is then converted into a digital signal. Then the digital signal is converted to analog signal, as shown in Figure 2c. These signals identify hand movements through codes, calculate the coordinate position of the cursor on the screen, the accelerometer and gyroscope data, convert the acceleration and angular speed into the displacement and Angle changes of the hand in space through integral operations, and then map the hand movements to the screen coordinate system according to the pre-set mapping relationship.[3] The hand gesture recognition algorithm is used to determine whether the hand is in a moving state, as well as the direction and speed of movement.[4] If the hand movement is detected, the target coordinate position of the cursor on the screen is calculated according to the cursor control algorithm, and the cursor movement instruction is sent to the computer through the communication interface. After receiving the instruction, the computer updates the cursor position to realize the movement of the cursor on the screen.[5]

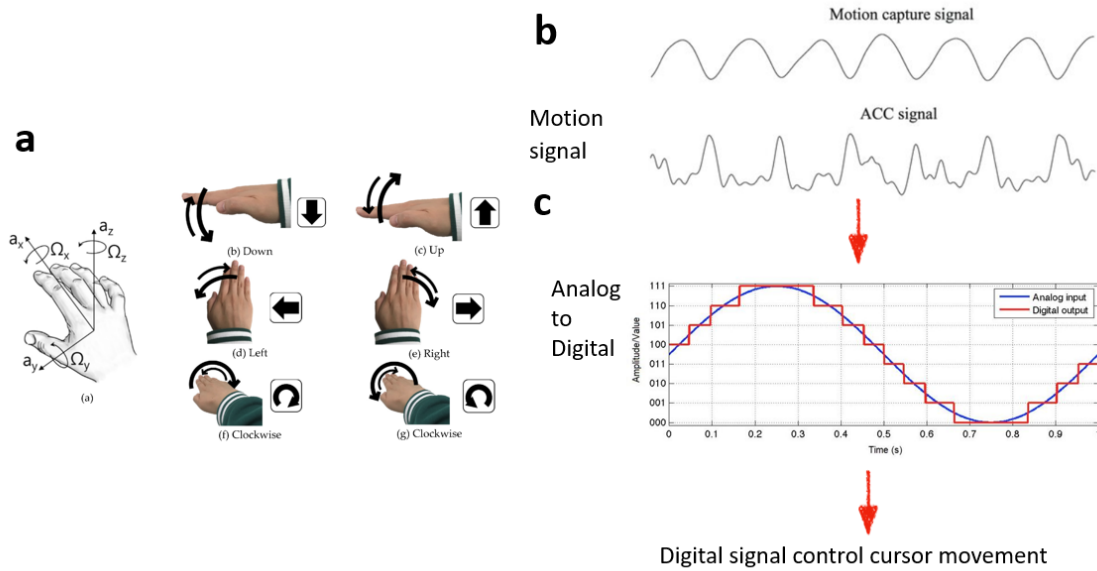


Figure 2: a, the principle of 9DS0 sensor. b, motion signal and c, analog signal to digital signal

3 Components

The project requires the following materials:

- Arduino Leonardo R3
- LSM9DS0 Sensor
- MPR121 Sensor
- laptop

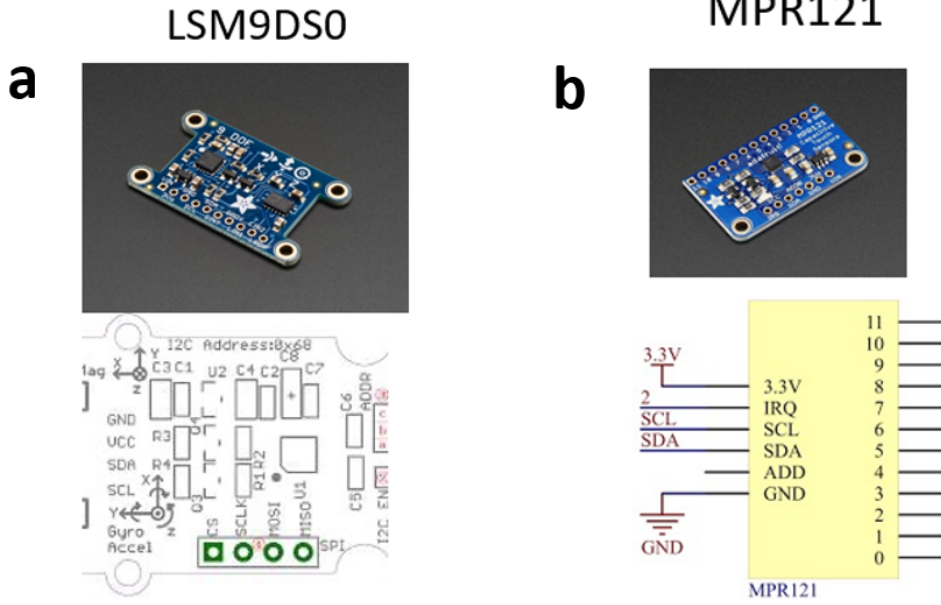


Figure 3: a, the LSM9DS0 sensor. b, the MPR121 sensor

The LSM9DS0 [6] is a 9-degree of freedom motion sensor that integrates a 3-axis accelerometer, 3-axis gyroscope and 3-axis magnetometer. It has three main parts, the accelerometer: it can sense how an object is accelerating, slowing down or coming to rest in three directions. Gyroscope: Used to detect the rotation speed of an object around three axes. Magnetometer: can detect the strength and direction of the surrounding magnetic field, mainly to determine the direction of the Earth's magnetic field, used in the electronic compass to help the device identify the direction. This sensor can combine these measurements to more accurately calculate the attitude of the object, and it communicates with other devices through a digital interface, power consumption is also relatively low, and the range can be selected according to needs.

The MPR121 [7] is a near-capacitive touch sensor controller designed and manufactured by NXP that is only $8\mu A$ at low speed scanning for portable and battery-powered devices. It can also handle 12 touch channels, which can be used for multi-touch keys or area detection, such as touch keyboards and panel making. Each sensor input can be automatically configured and calibrated to adapt to different touch environments, simplifying design. Its touch detection function enables when the human finger touches any electrode of the sensor, it can accurately detect the touch action, and output the corresponding signal, while the capacitor detection function determines the touch or proximity event by detecting the capacitance change, with high sensitivity and accuracy, and can detect small capacitance changes, so as to achieve accurate touch and proximity detection.

4 Making Process

4.1 Step 1: Design and Build the Glove Mouse

The circuit of the heartbeat sensor is shown in Figure 4. The circuit consists of four parts from left to right;

1. MPR121 sensor: Detects finger touch motion
2. ARDUINO Leonardo r3: It directly uses the USB communication function of ATmega32u4 and cancels the USB-to-UART chip. This allows Leonardo to be connected to a computer not only

as a virtual (CDC) serial/COM port, but also as a mouse or keyboard.

3. LSM9DS0: Can sense how the hand accelerates, decelerates or stops in three directions, and calculate the posture of the object more accurately
4. Button: used to control mouse clicks with fingers

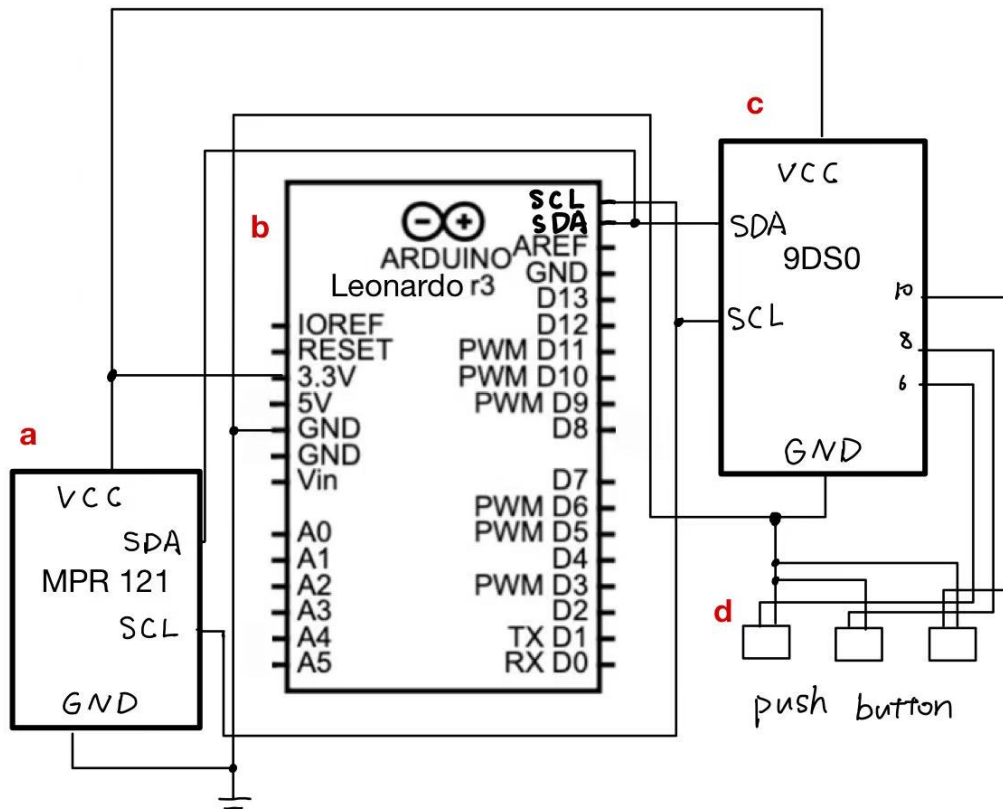


Figure 4: Circuit of the Glove Mouse, indicating the output of a, MPR121 sensor, b, Arduino Leonardo R3, c, 9DS0 sensor and d, the buttons.

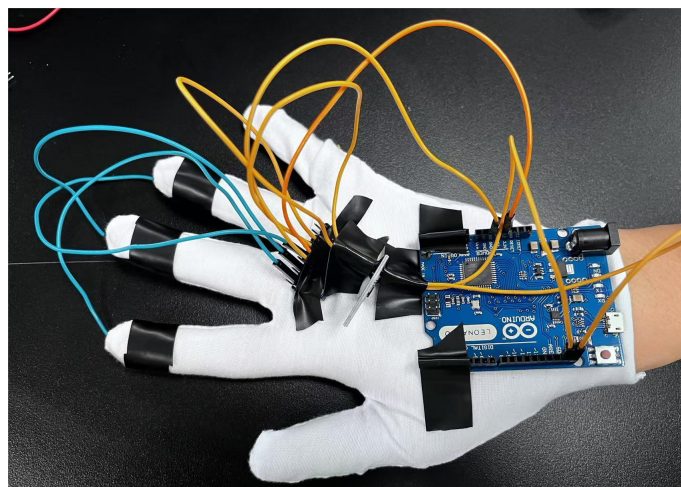
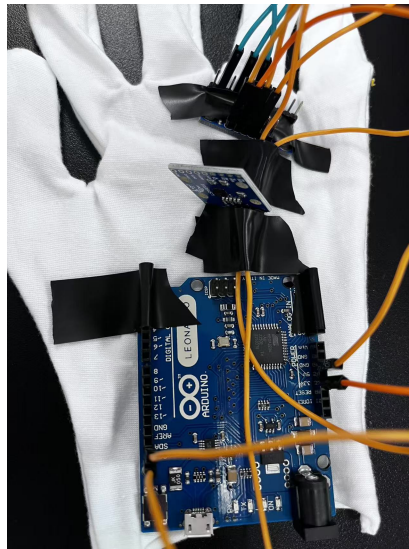


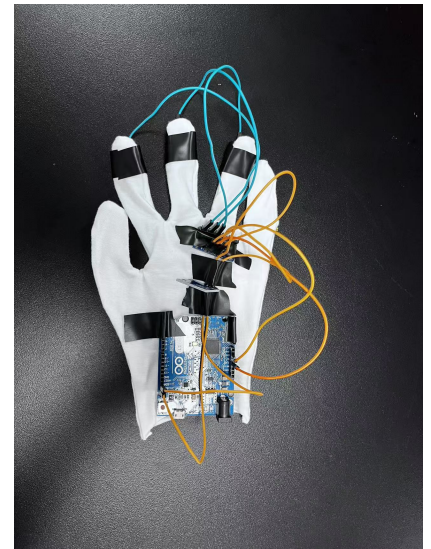
Figure 5: The real circuit of the Glove Mouse



(a) Bottom-down view of the glove mouse.



(b) Close-up view of the glove mouse.



(c) Up-down view of the glove mouse.

Figure 6: Different views of the wearable glove mouse.

Figure 6 shows the real device glove mouse for different views.

4.2 Step 2: Programming of the Arduino Simulate Mouse Movement and Keystroke Operating System

Arduino is programmed to control cursor movements and clicks. This program is designed as follows.

1. Initialize the sensor and configure its working mode. Set the initial value to define each pin
2. Collect and process sensor data by moving the mouse. Read the accelerometer X and Y axis data, map it to the mouse movement range, calculate the amount of mouse movement move the mouse.
3. Collect and process sensor data to control mouse buttons. Read the current state of the touch sensor and compare with the previous state, trigger the corresponding mouse button time to simulate the mouse button operation
4. Finally, the mouse moves the cursor and controls keys as the hand moves and the button is pressed

The program provides how to control the cursor to move on the computer screen. The code is given as follows and uploaded to Moodle.

Listing 1: Glove Mouse Code

```

1 #include <SPI.h>
2 #include <Wire.h>
3 #include <Adafruit_Sensor.h>
4 #include <Adafruit_LSM9DS0.h>
5 #include "Adafruit_MPR121.h"
6 #include "Mouse.h"
7
8 #define LSM9DS0_XM_CS 10

```



```

9  #define LSM9DS0_GYRO_CS 9
10 #define LSM9DS0_SCLK 13
11 #define LSM9DS0_MISO 12
12 #define LSM9DS0_MOSI 11
13
14 // LSM9DS0, use I2C, ID #1005
15 Adafruit_LSM9DS0 lsm = Adafruit_LSM9DS0(1005);
16
17 // MPR121, use I2C, ID #1000
18 Adafruit_MPR121 cap = Adafruit_MPR121();
19
20 float xNormal = -3.0,
21       yNormal = 1,
22       // Clamp inputs to this range (m/s^2)
23       accelMax = 20,
24       accelMin = -20,
25       // Map inputs to this range (pixels)
26       outMin   = 30,
27       outMax   = -30,
28       // How many milliseconds between polling the accelerometer
29       delayRate = 7;
30
31 const boolean DEBUG = true;
32
33 int calibrationCounter = 0,
34     calibrationRuns = 20;
35
36 float xRange[] = {0,0};
37 float yRange[] = {0,0};
38
39 const int MOUSE_LEFT_PIN = 10,
40          MOUSE_RIGHT_PIN = 6,
41          MOUSE_MID_PIN   = 8;
42
43 uint16_t lasttouched = 0,
44          currtouched = 0;
45
46
47 // Configures the gain and integration time for the TSL2561
48 void configureSensor(void) {
49     // Set the accelerometer range
50     lsm.setupAccel(lsm.LSM9DS0_ACCEL_RANGE_2G);
51     //lsm.setupAccel(lsm.LSM9DS0_ACCEL_RANGE_4G);
52     //lsm.setupAccel(lsm.LSM9DS0_ACCEL_RANGE_6G);
53     //lsm.setupAccel(lsm.LSM9DS0_ACCEL_RANGE_8G);
54     //lsm.setupAccel(lsm.LSM9DS0_ACCEL_RANGE_16G);
55
56     // Set the magnetometer sensitivity
57     lsm.setupMag(lsm.LSM9DS0_MAGGAIN_2GAUSS);
58     //lsm.setupMag(lsm.LSM9DS0_MAGGAIN_4GAUSS);

```

```

59 //lsm.setupMag(lsm.LSM9DS0_MAGGAIN_8GAUSS);
60 //lsm.setupMag(lsm.LSM9DS0_MAGGAIN_12GAUSS);
61
62 // Setup the gyroscope
63 lsm.setupGyro(lsm.LSM9DS0_GYROSCALE_245DPS);
64 //lsm.setupGyro(lsm.LSM9DS0_GYROSCALE_500DPS);
65 //lsm.setupGyro(lsm.LSM9DS0_GYROSCALE_2000DPS);
66 }
67
68
69 void setup(void) {
70     if (DEBUG) Serial.begin(9600);
71
72     dof_setup();
73     mpr_setup();
74 }
75
76
77 void loop(void) {
78     dof_loop();
79     mpr_loop();
80 }
81
82
83 void mpr_setup() {
84     // needed to keep leonardo/micro from starting too fast!
85     while (!Serial);
86     if (DEBUG) Serial.begin(9600); Serial.println("Adafruit MPR121
87         Capacitive Touch sensor test");
88
89     // Default address is 0x5A, if tied to 3.3V its 0x5B
90     // If tied to SDA its 0x5C and if SCL then 0x5D
91     if (!cap.begin(0x5A)) if (DEBUG) Serial.println("MPR121 not found"
92         );
93     else if (DEBUG) Serial.println("MPR121 found!");
94 }
95
96 void dof_setup() {
97     if(!lsm.begin()) { while(1); }
98
99     // Setup the sensor gain and integration time
100     configureSensor();
101
102     // Set a max and min value to not do anything
103     sensors_event_t accel, mag, gyro, temp;
104     lsm.getEvent(&accel, &mag, &gyro, &temp);
105 }
106

```

```

107 void dof_loop() {
108     /* Get a new sensor event */
109     sensors_event_t accel, mag, gyro, temp;
110     lsm.getEvent(&accel, &mag, &gyro, &temp);
111
112     float mx, my = 0.0;
113     float accelX = clamp(accel.acceleration.x, accelMin, accelMax);
114     float accelY = clamp(accel.acceleration.y, accelMin, accelMax);
115
116     // calibrate for the first five loops
117     if (calibrationCounter > calibrationRuns) {
118
119         if (accel.acceleration.x < xRange[0] || accel.acceleration.x >
120             xRange[1]) {
121             mx = map(accelY - yNormal, accelMin, accelMax, outMin, outMax);
122         }
123
124         if (accel.acceleration.y < yRange[0] || accel.acceleration.y >
125             yRange[1]) {
126             my = map(accelX - xNormal, accelMin, accelMax, outMin, outMax);
127         }
128
129         Mouse.move(mx, my, 0);
130     }
131     else {
132         calibrate(accel.acceleration.x, accel.acceleration.y);
133         calibrationCounter++;
134     }
135     delay(delayRate);
136 }
137
138 void mpr_loop() {
139     // Get the currently touched pads
140     currtouched = cap.touched();
141     checkTouch();
142     // reset our state
143     lasttouched = currtouched;
144 }
145
146 void checkTouch() {
147     for (uint8_t i = 0; i < 12; i++) {
148         // it if /is/ touched and /wasn't/ touched before, alert!
149         if ( (currtouched & _BV(i)) && !(lasttouched & _BV(i)) )
150             touchHandler(i);
151
152         // if it /was/ touched and now /isn't/, alert!
153         if ( !(currtouched & _BV(i)) && (lasttouched & _BV(i)) )
154             releaseHandler(i);
155     }
156 }

```



```

153 }
154 }
155
156
157 void calibrate(float x, float y) {
158     if (calibrationCounter == 0) {
159         if (DEBUG) Serial.print("First time");
160         xRange[0] = x;
161         xRange[1] = x;
162
163         yRange[0] = y;
164         yRange[1] = y;
165     }
166     else {
167         xRange[0] = min(xRange[0], x);
168         xRange[1] = max(xRange[1], x);
169
170         yRange[0] = min(yRange[0], y);
171         yRange[1] = max(yRange[1], y);
172     }
173
174     xNormal = (xRange[0] + xRange[1])/2;
175     yNormal = (yRange[0] + yRange[1])/2;
176
177     if (DEBUG) {
178         Serial.print("X Min: "); Serial.print(xRange[0]);
179         Serial.print(", Max: "); Serial.println(xRange[1]);
180         Serial.print(", Norm: "); Serial.println(xNormal);
181
182         Serial.print("Y Min: "); Serial.print(yRange[0]);
183         Serial.print(", Max: "); Serial.println(yRange[1]);
184         Serial.print(", Norm: "); Serial.println(yNormal);
185     }
186 }
187
188
189 void touchHandler(int id) {
190
191     switch(id) {
192         case MOUSE_LEFT_PIN:
193             if (DEBUG) Serial.println("Left button touched"); Serial.
                println("");
194             Mouse.press(MOUSE_LEFT);
195             break;
196         case MOUSE_MID_PIN:
197             if (DEBUG) Serial.println("Middle button touched"); Serial.
                println("");
198             Mouse.press(MOUSE_MIDDLE);
199             break;
200         case MOUSE_RIGHT_PIN:

```

```

201     if (DEBUG) Serial.println("Right button touched"); Serial.
        println("");
202     Mouse.press(MOUSE_RIGHT);
203     break;
204 }
205 }
206
207
208 void releaseHandler(int id) {
209
210     switch(id) {
211         case MOUSE_LEFT_PIN:
212             if (DEBUG) Serial.println("Left button released"); Serial.
                println("");
213             Mouse.release(MOUSE_LEFT);
214             break;
215         case MOUSE_MID_PIN:
216             if (DEBUG) Serial.println("Middle button released"); Serial.
                println("");
217             Mouse.release(MOUSE_MIDDLE);
218             break;
219         case MOUSE_RIGHT_PIN:
220             if (DEBUG) Serial.println("Right button released"); Serial.
                println("");
221             Mouse.release(MOUSE_RIGHT);
222             break;
223     }
224 }
225
226
227 float clamp(float value, float min_, float max_) {
228     return (value < min_) ? min_ : (value > max_) ? max_ : value;
229 }
230
231
232 void displaySensorDetails(int delayAmt) {
233     sensor_t accel, mag, gyro, temp;
234     lsm.getSensor(&accel, &mag, &gyro, &temp);
235
236     if (DEBUG) {
237         Serial.println(F("-----"));
238         Serial.print (F("Sensor:      ")); Serial.println(accel.name);
239         Serial.print (F("Driver Ver:  ")); Serial.println(accel.version
        );
240         Serial.print (F("Unique ID:   ")); Serial.println(accel.
            sensor_id);
241         Serial.print (F("Max Value:   ")); Serial.print(accel.max_value
            ); Serial.println(F(" m/s^2"));
242         Serial.print (F("Min Value:   ")); Serial.print(accel.min_value
            ); Serial.println(F(" m/s^2"));

```

```

243     Serial.print (F("Resolution:  ")); Serial.print(accel.
        resolution); Serial.println(F("  m/s^2"));
244     Serial.println(F("-----"));
245     Serial.println(F(""));
246 }
247 delay(delayAmt);
248 }

```

5 Result and Conclusion

Through this project, we can control the mouse to move and click with the palm control movement. A video of the result is attached to the PPT file of the presentation. In the future, we may be able to detangle the glove mouse from the USB cable via devices such as Bluetooth to make it more flexible

6 Contribution

- Xie Wen-hao
- Shi Zhi-bo

We proposed the idea and wrote the report.

References

- [1] Han, H., & Yoon, S. W. (2019). Gyroscope-Based Continuous Human Hand Gesture Recognition for Multi-Modal Wearable Input Device for Human Machine Interaction. *Sensors*, 19(11), 2562. <https://doi.org/10.3390/s19112562>
- [2] Paul, W. (n.d.). Mouse & Rat: Technical Explanation. Retrieved from <https://will-paul.com/posts/mouse-rat-technical-explanation/>
- [3] Behera, A., Kumar P, A., Badiger, A. S., Hemanth, & Jeyaseelan, V. (2023). WEARABLE GLOVE MOUSE. *IJARIIIE*, 9(3), 570-580. Retrieved from https://ijariie.com/AdminUploadPdf/Wearable_Glove_Mouse_ijariie20100.pdf?srsltid=AfmB0oqqY0Jc4KH_dVGSD8vFF1MWG8tatIMiRRX0dAuJTo04ZcE-5Qrn
- [4] Arduino Mouse Control Glove. (n.d.). Retrieved from <https://www.instructables.com/Arduino-Mouse-Control-Glove/>
- [5] Gesture Mouse - Controlling Computer With a Glove. (n.d.). Retrieved from <https://www.instructables.com/Gesture-Mouse-Controlling-Computer-With-a-Glove/>
- [6] Datasheet of LSM9DS0, now <https://www.alldatasheet.com/view.jsp?Searchword=LSM9DS0>
- [7] Datasheet of MPR121, now <https://www.alldatasheet.com/view.jsp?Searchword=MPR121>